Contents

1 Dynamic Rupture Implementation Plan v4 (revised)

Date: 2026-04-12 (revised after code review) **Status:** Draft — addresses R-001 through R-010 from `REVIEW.md` **Predecessor:** v4 original (dual-inheritance conflict, bracket sign error, SBI dimension mismatch) **Benchmark:** SCEC TPV102, then BP5 hybrid QD+dynamic **References:**

- Dumbser & Käser (2006), ADER-DG for 3D elastic waves
- de la Puente et al. (2009), fault Riemann solver for dynamic rupture
- Pelties et al. (2014), SeisSol fault coupling (eq. A2)
- Uphoff (2020), PhD dissertation, Chapter 4 (eq. 4.51–4.60)
- Komatitsch & Martin (2007), unsplit convolutional PML
- Lapusta et al. (2000), spectral BIE for earthquake sequences
- SeisSol source code `/Users/chunhuizhao/projects/SeisSol`
- SCEC TPV101/102 benchmark specification

1.1 Changes from v3

#	v3 Gap	v4 Resolution
1	Only first-order absorbing BC	**Three boundary methods: ABC (Phase 2a), PML (Phase 2b), SBI (Phase 2c)**. For BP5 hybrid, PML or SBI eliminates long-time reflection contamination.
2	Trial-and-correction traction lacked motivation	**New Section 4.1: full pedagogical derivation** — why trial traction, what problem it solves, step-by-step logic
3	Imposed state (Uphoff 4.60) unexplained	**New Section 4.3: physical explanation** — why the DG flux needs an imposed state, how it connects to the Godunov solver
4	Equation indices didn't match code variables	**Every equation now has a dictionary** mapping math symbols to code variables, with restated definitions
5	TPV102 had no local-vs-cluster testing strategy	**Phase 4 split into 4a (local verification) and 4b (Frontera benchmark)** with sbatch templates
6	QD↔FD transfer lacked speed/artifact analysis	**Phase 5 expanded** with CG→DG projection cost, numerical artifact mitigation, warm-start strategy
7	Only Brent for friction solver	**Dual solver: Brent (CPU default) + Newton-Raphson (GPU default)** with detailed comparison

---

1.2 Review Fixes (R-001 through R-010)

Review ID	Fix Applied
R-001 (CRITICAL)	**WaveOperator no longer inherits DomainOperator.** Inherits only `TimeDependentOperator`. Uses `FaultBasis`/`FaultGeometry` via composition. New `SEASDynamicOperator` wraps `WaveOperator*` + `FaultFaceFlux*`, following the same composition pattern as `SEASQuasiDynamicOperator`.
R-002 (CRITICAL)	**Bracket signs corrected.** $g(0) = -\Theta < 0$ and $g(\Theta/\eta_s) = +\lvert\sigma_n\rvert f > 0$. Added note about polarity difference from QD Brent solver.
R-003 (MODERATE)	**SBI DtN kernel corrected to 2D Fourier.** Eq. (19) now uses $\mathcal{F}_{2D}^{-1}$ with $\lvert k\rvert = \sqrt{k_y^2 + k_z^2}$. LOC estimate for `sbi_kernel.cpp` increased to ~300.
R-004 (MODERATE)	**PML corner code fixed to per-component damping.** `ComputeDamping()` returns 3 directional values (d_x, d_y, d_z). Per-component: $d_c = d_x D_x[c] + d_y D_y[c] + d_z D_z[c]$.
R-005 (MODERATE)	**Fault-local → global rotation step explicitly documented.** New subsection in Section 4.4 documents the 4-step rotation pipeline using `FaultBasis`. Code dictionary updated to use fault-local indices, not global enums.
R-006 (MODERATE)	**Frontera sbatch template replaced** with production pattern from `jobs/bp5/`. Uses `module load` + `LD_LIBRARY_PATH`, no conda.
R-007 (MODERATE)	**TPV102 driver CLI matches `seas_bp5_full` pattern** with `--flag value` style, not positional TOML.
R-008 (MODERATE)	**Fault-tip velocity taper added.** New "Problem 4" in Section 5.2 applies Gaussian taper over ~3 elements near fault tips using `FaultGeometry::GetFaultCoords2D()`.

Review ID	Fix Applied
R-009 (LOW)	**FD→QD transfer detailed steps added.** 5-step re-initialization sequence with equilibrium verification.
R-010 (LOW)	**Test count table clarified** with per-file breakdown.

1.3 Overview

Build a 3D dynamic rupture DG code within the SEAS-MFEM project. **Architecture (R-001 fix):** `WaveOperator` inherits **only** `TimeDependentOperator` (not `DomainOperator`), following the same composition pattern as `SEASQuasiDynamicOperator`. It owns its own mesh, L2 FE space, mass matrix, and Godunov flux. Shared fault-coupling code (`FaultBasis`, `FaultGeometry`, `DieterichRuinaFriction`) is accessed via **composition** — `WaveOperator` constructs and owns a `FaultBasis` and exposes accessors, but does not inherit the `DomainOperator` interface whose `Solve()`/`ComputeTraction()` methods are designed for implicit quasi-static solves.

A separate `SEASDynamicOperator` (analogous to `SEASQuasiDynamicOperator`) wraps `WaveOperator*` + `FaultFaceFlux*` and provides the coupled `Mult()`. For hybrid QD↔FD (Phase 5), the `SEASHybridOperator` switches between composed `SEASQuasiDynamicOperator*` and `SEASDynamicOperator*` — trivial pointer swap, no type casting.

Three boundary methods (ABC, PML, SBI) and a dual friction solver (Brent on CPU, Newton-Raphson on GPU) complete the design.

Estimated new code: ~5,500 LOC.

1.4 Constraints

- **Interface constraints:** `DomainOperator`, `ConstitutiveModel`, `FaultBasis`, `FaultGeometry`, `DieterichRuinaFriction` interfaces unchanged.
- **Dependency constraints:** MFEM `L2_FECollection`, `ODESolver`, `ParMesh`. FFTW3 for SBI (optional, Phase 2c only).
- **Convention constraints:** Module-per-directory (`dynamic/`), test-per-feature, TOML config, Makefile integration.
- **Numerical constraints:** CFL-limited explicit stepping. Brent solver convergence $|g| < 10^{-8}$. Energy conservation at fault interface.

1.5 Reuse Map

(Same as v3 — see v3 Section "Reuse Map" for full table.)

---

2 PART I: MATHEMATICAL FOUNDATIONS

2.1 1. Governing Equations (Strong Form)

2.1.1 1.1 Velocity-Stress System

The 3D isotropic linear elastic wave equation in first-order velocity-stress form.

State vector (9 unknowns):

$$\mathbf{Q} = \begin{pmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{xz} \\ v_x \\ v_y \\ v_z \end{pmatrix}$$

Code index convention:

Math symbol	Code enum	Index	Physical meaning
σ_{xx}	`SXX`	0	Normal stress (x-face, x-direction)
σ_{yy}	`SYY`	1	Normal stress (y-face, y-direction)
σ_{zz}	`SZZ`	2	Normal stress (z-face, z-direction)
σ_{xy}	`SXY`	3	Shear stress (x-face, y-direction)
σ_{yz}	`SYZ`	4	Shear stress (y-face, z-direction)
σ_{xz}	`SXZ`	5	Shear stress (x-face, z-direction)
v_x	`VX`	6	Particle velocity (x-direction)
v_y	`VY`	7	Particle velocity (y-direction)
v_z	`VZ`	8	Particle velocity (z-direction)

Conservation law:

$$\frac{\partial \mathbf{Q}}{\partial t} + \mathbf{A}\frac{\partial \mathbf{Q}}{\partial x} + \mathbf{B}\frac{\partial \mathbf{Q}}{\partial y} + \mathbf{C}\frac{\partial \mathbf{Q}}{\partial z} = \mathbf{0} \tag{1}$$

where $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ are 9×9 Jacobian matrices and:

- λ, μ = Lamé parameters (Pa). Code: `lambda_`, `mu_`
- ρ = density (kg/m³). Code: `rho_`
- $c_p = \sqrt{(\lambda + 2\mu)/\rho}$ = P-wave speed (m/s). Code: `cp_`
- $c_s = \sqrt{\mu/\rho}$ = S-wave speed (m/s). Code: `cs_`

2.1.2 1.2 Jacobian Matrices

A-matrix (x-direction flux), from SeisSol `getTransposedCoefficientMatrix()` (`ElasticSetup.h:28–77`):

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & -(\lambda+2\mu) & -\lambda & -\lambda \\ 0 & 0 & 0 & 0 & 0 & 0 & -\lambda & -(\lambda+2\mu) & -\lambda \\ 0 & 0 & 0 & 0 & 0 & 0 & -\lambda & -\lambda & -(\lambda+2\mu) \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\mu & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\mu \\ 0 & 0 & 0 & 0 & 0 & 0 & -\mu & 0 & 0 \\ -\frac{1}{\rho} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\frac{1}{\rho} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{1}{\rho} & 0 & 0 & 0 \end{pmatrix}$$

Code mapping for A: `A[SXX][VX] = -(lambda+2mu)`, `A[VX][SXX] = -1/rho`, etc.

B, **C** follow by cyclic coordinate permutation (see v3 Section 1.2).

2.1.3 1.3 Eigenstructure

$$\mathbf{A}_n = n_x\mathbf{A} + n_y\mathbf{B} + n_z\mathbf{C} = \mathbf{R}\,\mathbf{R}^{-1}$$

$$= \mathrm{diag}\big(+c_p,\ +c_s,\ +c_s,\ 0,\ 0,\ 0,\ -c_s,\ -c_s,\ -c_p\big)$$

Eigenvector matrix $\mathbf{R}$ given in v3 Section 1.3.

2.1.4 1.4 Rotation to Face-Normal Coordinates

$$\mathbf{T} = \begin{pmatrix} \mathbf{T}_\sigma & \mathbf{0} \\ \mathbf{0} & \mathbf{T}_v \end{pmatrix}_{9\times 9}, \qquad \mathbf{Q}_{\mathrm{rot}} = \mathbf{T}^{-1}\mathbf{Q}$$

Detailed matrices given in v3 Section 1.4.

2.2 2. DG Weak Form Derivation

2.2.1 2.1 From Strong Form to Weak Form

Starting point: Eq. (1) in element T_e (subscript e = element index).

Step 1 — Multiply by test function and integrate:

$$\int_{T_e} \Phi_k(\mathbf{x})\,\frac{\partial \mathbf{Q}}{\partial t}\,dV + \int_{T_e} \Phi_k(\mathbf{x})\Big(\mathbf{A}\frac{\partial \mathbf{Q}}{\partial x} + \mathbf{B}\frac{\partial \mathbf{Q}}{\partial y} + \mathbf{C}\frac{\partial \mathbf{Q}}{\partial z}\Big)\,dV = 0$$

where:

- $\Phi_k(\mathbf{x})$ = degree-N polynomial basis function, $k = 1, \ldots, n_{\mathrm{dof}}$. Code: `fe->CalcShape(ip, shape)`
- T_e = element e with n_{dof} DOFs per component. For hexahedra: $n_{\mathrm{dof}} = (N+1)^3$
- $\mathbf{Q}$ is approximated as $\mathbf{Q}_h = \sum_{l=1}^{n_{\mathrm{dof}}} \hat{\mathbf{Q}}_l(t)\,\Phi_l(\mathbf{x})$

Step 2 — Integration by parts (divergence theorem on the flux terms):

$$\underbrace{\int_{T_e} \Phi_k\,\frac{\partial \mathbf{Q}}{\partial t}\,dV}_{\text{mass term}} + \underbrace{\oint_{\partial T_e} \Phi_k\,\mathbf{F}_n^h\,dS}_{\text{face flux}} - \underbrace{\int_{T_e} \frac{\partial \Phi_k}{\partial x_j}\,\mathbf{F}_j(\mathbf{Q})\,dV}_{\text{volume integral}} = 0 \tag{2}$$

where:

- $\mathbf{F}_j(\mathbf{Q}) = \mathbf{A}_j\,\mathbf{Q}$ = physical flux in direction j ($\mathbf{A}_1 = \mathbf{A}$, $\mathbf{A}_2 = \mathbf{B}$, $\mathbf{A}_3 = \mathbf{C}$)
- $\mathbf{F}_n^h$ = numerical flux at face (Godunov upwind, Eq. 4/5/6)
- ∂T_e = boundary of element e (union of faces)
- Repeated index j summed over $\{x, y, z\} = \{1, 2, 3\}$

2.2.2 2.2 Matrix Form

Mass matrix ($n_{\mathrm{dof}} \times n_{\mathrm{dof}}$, one per element, same for all 9 components):

$$M_{kl}^{(e)} = \int_{T_e} \Phi_k\,\Phi_l\,dV \tag{2a}$$

Code: assembled via `MassIntegrator` or manually. Stored as `elem_mass_inv_[e]` (the inverse $(\mathbf{M}^{(e)})^{-1}$).

Volume integral (per element e, component q, DOF k):

$$\mathrm{Vol}_k^{q,(e)} = \sum_{l=1}^{n_{\mathrm{dof}}} \sum_{j=1}^{3} S_{kl}^{j,(e)} \, [\mathbf{A}_j]_{qr} \, \hat{Q}_l^{r,(e)} \tag{2b}$$

where the stiffness-like matrix is:

$$S_{kl}^{j,(e)} = \int_{T_e} \frac{\partial \Phi_k}{\partial x_j} \, \Phi_l \, dV$$

Code: `fe->CalcPhysDShape(*Tr, dshape)` gives $\partial \Phi_k / \partial x_j$ at quadrature points.

Quadrature approximation of Eq. (2b): at quadrature point $\mathbf{x}_q$ with weight w_q:

$$\mathrm{Vol}_k^{q,(e)} \approx \sum_{q=1}^{n_{\mathrm{qp}}} w_q \, |\det J_e(\mathbf{x}_q)| \, \left. \frac{\partial \Phi_k}{\partial x_j} \right|_{\mathbf{x}_q} \cdot \left[\mathbf{A}_j \, \mathbf{Q}_h(\mathbf{x}_q) \right]^q \tag{2c}$$

Code-to-math dictionary for Eq. (2c):

Math	Code variable	Type	Meaning
e	loop variable `e`	`int`	Element index, $0 \le e < n_e$
q (quadrature)	loop variable `q`	`int`	Quadrature point index, $0 \le q < n_{\mathrm{qp}}$
q (component)	loop variable `c`	`int`	State component, $0 \le c < 9$
k	loop variable `i`	`int`	DOF index, $0 \le i < n_{\mathrm{dof}}$
$w_q \, \lvert \det J \rvert$	`w = ip.weight * Tr->Weight()`	`real_t`	Weighted Jacobian determinant
$\partial \Phi_k / \partial x_j$	`dshape(i, j)`	`real_t`	Physical gradient of basis function k in direction j
$[\mathbf{A}_j \, \mathbf{Q}_h]^c$	`F[j][c]`	`real_t`	Component c of flux $\mathbf{A}_j \, \mathbf{Q}$ in direction j
$\hat{Q}_l^r$	`Q_e[c * ndof + i]`	`real_t`	Element DOF: component c, local DOF i

Volume integral code (implements Eq. 2c):

```
// For each element e:
for (int q = 0; q < nqp; q++) {          // quadrature point index
   real_t w = ip.weight * Tr->Weight();   // w_q * |det J|
   // Evaluate Q_h(x_q) by interpolating from element DOFs:
   // Q_qp[c] = sum_i shape(i) * Q_e[c * ndof + i]   for c = 0..8
   // Compute fluxes: F[j][c] = (A_j * Q_qp)[c]     for j = 0,1,2
   for (int c = 0; c < 9; c++)            // state component
      for (int i = 0; i < ndof; i++)      // DOF index
         for (int j = 0; j < 3; j++)      // spatial direction
            rhs_e[c * ndof + i] += w * dshape(i, j) * F[j][c];
}
```

Face flux (per face f, quadrature point q_f):

$$\text{Face}_k^{q,(e)} = \sum_{q_f=1}^{n_{\text{qpf}}} w_{q_f} |J_f| \, \Phi_k(\mathbf{x}_{q_f}) \, [\mathbf{F}_n^h(\mathbf{x}_{q_f})]^q \tag{2d}$$

Code-to-math dictionary for Eq. (2d):

Math	Code variable	Meaning
f	loop variable `f`	Face index
q_f	loop variable `q` in face loop	Face quadrature point
$w_{q_f} \lvert J_f \rvert$	`w = ip.weight * face_area`	Face weight times face Jacobian
$\Phi_k(\mathbf{x}_{q_f})$	`shape1(i)`	Basis function of Elem1 evaluated at face QP
$[\mathbf{F}_n^h]^c$	`F_h[c]`	Component c of numerical flux
$\mathbf{n}$	`nor` (unit normal)	Outward normal from Elem1

Semi-discrete ODE (per element):

$$\frac{d\hat{\mathbf{Q}}^{(e)}}{dt} = \left(\mathbf{M}^{(e)}\right)^{-1}\left(-\text{Face}^{(e)} + \text{Vol}^{(e)}\right) \tag{3}$$

Code: `WaveOperator::Mult(Q, dQdt)` computes the RHS.

2.3 3. Godunov Upwind Flux

2.3.1 3.1 Interior Faces

$$\boxed{\mathbf{F}_n^h = \mathbf{A}_n^+ \, \mathbf{Q}_{\text{self}} + \mathbf{A}_n^- \, \mathbf{Q}_{\text{nbr}}} \tag{4}$$

where $\mathbf{A}_n^{\pm} = \frac{1}{2}(\mathbf{A}_n \pm |\mathbf{A}_n|)$ and $|\mathbf{A}_n| = \mathbf{R} \, |\mathbf{\Lambda}| \, \mathbf{R}^{-1}$.

Code: `flux_.Interior(nor, Q_self, Q_nbr, F_h)` where `nor[3]` is the unit outward normal from Elem1, `Q_self[9]` is Elem1's state, `Q_nbr[9]` is Elem2's state, and `F_h[9]` is the output flux.

2.3.2 3.2 Absorbing BC (First-Order)

$$\boxed{\mathbf{F}_n^{\text{abs}} = \mathbf{A}_n^+ \, \mathbf{Q}_{\text{self}}} \tag{5}$$

Code: `flux_.Absorbing(nor, Q_self, F_h)`.

Limitation: Reflection coefficient at incidence angle θ: $R(\theta) \sim O(\sin^2\theta)$. At 30°: $R \approx$ 15-30%. At normal incidence: $R \approx 0\%$. **Unacceptable for BP5 hybrid** (100+ year simulations with repeated earthquakes).

2.3.3 3.3 Free Surface

$$\boxed{\mathbf{F}_n^{\text{free}} = \mathbf{A}_n^+ \, \mathbf{Q}_{\text{self}} + \mathbf{A}_n^- \, \Gamma \, \mathbf{Q}_{\text{self}}} \tag{6}$$

where $\Gamma = \text{diag}(-1, +1, +1, -1, +1, -1, +1, +1, +1)$ in rotated coordinates mirrors stress components with an odd number of normal indices. This enforces $\sigma \cdot \mathbf{n} = \mathbf{0}$.

Code: `flux_.FreeSurface(nor, Q_self, F_h)`.

2.4 4. Fault Riemann Solver — Detailed Derivation

2.4.1 4.1 Why Trial-and-Correction? (The Physical Picture)

The fundamental problem at a fault face: At each time step, the DG method needs to compute a numerical flux $\mathbf{F}_n^h$ at the fault face, just as it does at interior faces. But unlike interior faces, the fault has *friction* that constrains the traction and slip rate. We can't just apply the Godunov flux (Eq. 4) because it would treat the fault as a welded interface (no slip).

The solution is a two-step process:

1. **Trial step (pretend the fault is locked):** Compute what the traction *would be* if the fault were welded shut (zero slip). This "trial traction" τ^{trial} is the Godunov state assuming perfect coupling — it combines information from waves arriving from both sides.
2. **Correction step (apply friction):** The trial traction is generally too high — friction can't sustain it. Solve the friction law to find the actual slip rate $\hat{V}$ and compute the "corrected traction" $\tau^{\text{corr}} = \tau^{\text{trial}} - \eta_s \mathbf{V}$, where the correction term $\eta_s \mathbf{V}$ represents the traction reduction due to fault slip.

Why this works (physical intuition):

- The trial traction represents the *elastic demand*: what the surrounding medium wants to impose on the fault.
- The friction law determines the *fault strength*: the maximum traction the fault can sustain.
- When demand exceeds strength, the fault slips, and the correction reduces the traction to the frictional strength level.
- The correction $\eta_s \mathbf{V}$ is proportional to slip rate because faster slip radiates more energy into the medium, reducing the traction.

Why not solve the coupled system directly? The trial-and-correction approach *is* the exact solution to the coupled wave-friction Riemann problem. It decomposes the problem into:

1. A **linear** Godunov problem (trial traction — fast, closed-form)
2. A **nonlinear** scalar friction equation (correction — requires iterative solver)

This is much more efficient than solving the full nonlinear 9-component Riemann problem.

2.4.2 4.2 Trial Traction: Derivation from the Godunov State

Setup: Consider a fault face with normal $\mathbf{n}$ separating elements "+" and "−". In fault-local coordinates (normal n, tangent t_1, tangent t_2), the state on each side is:

$$\mathbf{Q}^{\pm} = \left(\sigma_n^{\pm},\ \sigma_{t_1 t_1}^{\pm},\ \sigma_{t_2 t_2}^{\pm},\ \tau_1^{\pm},\ \tau_{t_1 t_2}^{\pm},\ \tau_2^{\pm},\ v_n^{\pm},\ v_{t_1}^{\pm},\ v_{t_2}^{\pm}\right)$$

Code-to-math dictionary (fault-local frame):

Math	Code (after rotation to fault-local)	Meaning
$\sigma_n^{\pm}$	`Q_plus[SXX]`, `Q_minus[SXX]`	Normal stress on ± side
$\tau_1^{\pm}$	`Q_plus[SXY]`, `Q_minus[SXY]`	Tangent-1 (dip) traction
$\tau_2^{\pm}$	`Q_plus[SXZ]`, `Q_minus[SXZ]`	Tangent-2 (strike) traction
$v_n^{\pm}$	`Q_plus[VX]`, `Q_minus[VX]`	Normal velocity
$v_{t_1}^{\pm}$	`Q_plus[VY]`, `Q_minus[VY]`	Tangent-1 velocity
$v_{t_2}^{\pm}$	`Q_plus[VZ]`, `Q_minus[VZ]`	Tangent-2 velocity

Impedances (characterize the "stiffness" of the medium on each side):

$$Z_p^{\pm} = \rho^{\pm} c_p^{\pm} \quad \text{(P-impedance)}, \qquad Z_s^{\pm} = \rho^{\pm} c_s^{\pm} \quad \text{(S-impedance)}$$

$$\eta_p = \frac{Z_p^+ Z_p^-}{Z_p^+ + Z_p^-} \quad \text{(harmonic mean)}, \qquad \eta_s = \frac{Z_s^+ Z_s^-}{Z_s^+ + Z_s^-}$$

For homogeneous material ($\rho^+ = \rho^-$): $\eta_p = Z_p/2 = \rho\, c_p/2$ and $\eta_s = Z_s/2 = \rho\, c_s/2 = \mu/(2c_s)$.

Code: `data.eta_p`, `data.eta_s`, `data.Zp_plus`, `data.Zs_minus`, etc.

Derivation of Eq. (7): Start from the Godunov compatibility conditions at a locked interface. For a P-wave arriving from the + side, the characteristic relation is:

$$\sigma_n^* - \sigma_n^+ = -Z_p^+ \,(v_n^* - v_n^+)$$

From the − side:

$$\sigma_n^* - \sigma_n^- = +Z_p^- \,(v_n^* - v_n^-)$$

For a locked fault, $v_n^* = v_n^{*,+} = v_n^{*,-}$ (no normal opening). Solving these two equations for σ_n^*:

$$\sigma_n^{\text{trial}} = \frac{Z_p^- \sigma_n^+ + Z_p^+ \sigma_n^- + Z_p^+ Z_p^- (v_n^- - v_n^+)}{Z_p^+ + Z_p^-}$$

Rewriting with η_p:

$$\boxed{\sigma_n^{\text{trial}} = \eta_p \left(v_n^- - v_n^+ + \frac{\sigma_n^+}{Z_p^+} + \frac{\sigma_n^-}{Z_p^-} \right)} \tag{7a}$$

The same derivation for S-waves (using Z_s instead of Z_p) gives:

$$\boxed{\tau_1^{\text{trial}} = \eta_s \left(v_{t_1}^- - v_{t_1}^+ + \frac{\tau_1^+}{Z_s^+} + \frac{\tau_1^-}{Z_s^-} \right)} \tag{7b}$$

$$\boxed{\tau_2^{\text{trial}} = \eta_s \left(v_{t_2}^- - v_{t_2}^+ + \frac{\tau_2^+}{Z_s^+} + \frac{\tau_2^-}{Z_s^-} \right)} \tag{7c}$$

Physical interpretation of Eq. (7): The trial traction is an impedance-weighted average of the tractions and velocities from both sides. If both sides have identical states, the trial traction equals the existing traction (no change). If there's a velocity contrast ($v^- \neq v^+$), the trial traction increases — the medium is trying to "push" the fault.

Code (in `FaultFaceFlux::ComputeTrialTraction`):

```
real_t invZp_plus  = 1.0 / data.Zp_plus;   // 1/Z_p^+
real_t invZs_minus = 1.0 / data.Zs_minus;   // 1/Z_s^-
// Eq. (7a):
sigma_n_trial = data.eta_p * (Q_minus[VX] - Q_plus[VX]
                            + Q_plus[SXX] * invZp_plus
                            + Q_minus[SXX] * invZp_minus);
```

2.4.3 4.3 Friction Solve (The Correction)

Total traction = pre-stress + trial (perturbation from wave propagation):

$$\sigma_n^{\text{total}} = \sigma_{n,0} + \sigma_n^{\text{trial}}, \qquad \Theta = \sqrt{(\tau_{1,0} + \tau_1^{\text{trial}})^2 + (\tau_{2,0} + \tau_2^{\text{trial}})^2}$$

Code: `sigma_n_total = data.sigma_n0 + sigma_n_trial`, `Theta = sqrt(...)`.

Friction balance (same form as quasi-dynamic):

$$\boxed{\Theta = |\sigma_n^{\text{total}}|\, f(\hat{V}, \psi) + \eta_s\, \hat{V}} \tag{8}$$

where $f(\hat{V}, \psi) = a\ \text{arcsinh}\left[\frac{\hat{V}}{2V_0} \exp(\psi/a)\right]$ is the regularized rate-and-state friction coefficient, and $\hat{V}$ is the slip rate magnitude to solve for.

Why $\eta_s\, \hat{V}$ appears — and why it is NOT an approximation in DR:

In the quasi-dynamic (QD) formulation, the term $\eta\, V$ is an *approximation*: it replaces the full wave equation with a scalar damping term that approximates radiation from the fault (Rice, 1993). The approximation is $\eta = \mu/(2c_s)$.

In the fully-dynamic (DR) formulation, $\eta_s\, \hat{V}$ **is exact, not an approximation.** It arises naturally from the characteristic structure of the Riemann problem at the fault interface. Here is the precise chain:

1. The trial traction Θ (Eq. 7) is the locked-fault Godunov state — the traction that would exist if slip rate were zero.
2. When the fault slips at rate $\hat{V}$, it creates a velocity discontinuity across the interface.
3. By the characteristic compatibility relations of the elastic wave equation (the same relations that produced Eq. 7), this velocity discontinuity reduces the traction by exactly $\eta_s\, \hat{V}$.
4. Therefore: actual traction = $\Theta - \eta_s\, \hat{V}$, which rearranges to Eq. (8).

The harmonic mean impedance $\eta_s = Z_s^+ Z_s^- / (Z_s^+ + Z_s^-)$ emerges from the characteristic decomposition of the two-sided Riemann problem, not from any radiation approximation.

SeisSol code evidence (verifying this is exact, not approximate): - `FrictionSolverCommon.h:186–192`: Trial traction computed as impedance-weighted Godunov state using `etaS`, `invZs`, `invZsNeig` — these are exact impedance parameters, not approximations - `RateAndState.h:274–279`: Newton solver solves $g = (1/\eta_s)(\sigma_n\, \mu - \Theta) - \hat{s} = 0$ — this is Uphoff Eq. 4.57, derived from the exact Riemann problem (Uphoff 2020, Section 4.3.2, pp. 47-49) - `RateAndState.h:234–240`: Corrected traction computed as `traction1 = faultStresses.traction1 - etaS * slipRate1` — direct application of $\tau^{\text{corr}} = \tau^{\text{trial}} - \eta_s V$ - `CellLocalMatrices.cpp:695–699`: η_s computed as harmonic mean: `etaS = 1.0 / (1.0/zs + 1.0/zsNeig)`

The mathematical coincidence: For homogeneous material, $\eta_s = Z_s/2 = \rho c_s/2 = \mu/(2c_s) = \eta_{\text{QD}}$. The QD approximation *happens to give the exact impedance* for the homogeneous case. This is why `SolveSlipRateVectorPsi()` works for both QD and DR — the function solves the same equation $\Theta = |\sigma_n| f(V, \psi) + \eta V$ regardless of whether η came from a QD approximation or exact DR impedance.

Slip rate decomposition:

$$\text{strength} = |\sigma_n^{\text{total}}|\, f(\hat{V}, \psi)$$

$$V_1 = \hat{V}\, \frac{\tau_{1,0} + \tau_1^{\text{trial}}}{\text{strength} + \eta_s\, \hat{V}}, \qquad V_2 = \hat{V}\, \frac{\tau_{2,0} + \tau_2^{\text{trial}}}{\text{strength} + \eta_s\, \hat{V}} \tag{9}$$

Corrected traction (the actual traction the fault sustains):

$$\tau_1^{\text{corr}} = \tau_1^{\text{trial}} - \eta_s V_1, \qquad \tau_2^{\text{corr}} = \tau_2^{\text{trial}} - \eta_s V_2 \tag{10}$$

The correction $\eta_s V_i$ subtracts the radiation damping contribution. When $V = 0$ (locked fault), $\tau^{\text{corr}} = \tau^{\text{trial}}$ — no correction needed.

2.4.4 4.4 Imposed State: Why We Need It and How It Works

The problem: After solving for the slip rate and corrected traction, we have the *fault-interface* values. But the DG flux (Eq. 2d) needs **element-face** values — the state that each element "sees" at the fault boundary. These are the "imposed states" $\mathbf{Q}^{+,\text{imp}}$ and $\mathbf{Q}^{-,\text{imp}}$.

Why not just use the corrected traction directly? Because the DG scheme updates the *full state* (all 9 components) on each side of the face. The numerical flux $\mathbf{F}_n^h$ operates on complete state vectors, not just traction components. We need to construct a complete state on each side that is:

1. **Consistent** with the corrected traction (traction continuity: $\tau^{+,\text{imp}} = \tau^{-,\text{imp}} = \tau^{\text{corr}}$)
2. **Consistent** with the characteristic relations from each side (no spurious reflections)
3. **Has the correct velocity jump** ($v^{+,\text{imp}} - v^{-,\text{imp}} = V$, the slip rate)

Derivation of Eq. (11)-(12): From the same characteristic relations used for the trial traction, but now the fault is *slipping* (traction known, velocity unknown):

From the + side: $\tau^{\text{corr}} - \tau^{+} = Z_s^{+}\,(v_t^{+,\text{imp}} - v_t^{+})$

Solving for the imposed velocity:

$$v_{t_1}^{+,\text{imp}} = v_{t_1}^{+} + \frac{1}{Z_s^{+}}\left(\tau_1^{\text{corr}} - \tau_1^{+}\right) \tag{12b}$$

From the − side: $\tau^{\text{corr}} - \tau^{-} = -Z_s^{-}\,(v_t^{-,\text{imp}} - v_t^{-})$

$$v_{t_1}^{-,\text{imp}} = v_{t_1}^{-} - \frac{1}{Z_s^{-}}(\tau_1^{\text{corr}} - \tau_1^{-}) \tag{11b}$$

The full imposed state equations (Uphoff Eq. 4.60):

Minus side ($\mathbf{Q}^{-}$):

$$v_n^{-,\text{imp}} = v_n^{-} - \frac{1}{Z_p^{-}}(\sigma_n^{\text{corr}} - \sigma_n^{-}) \tag{11a}$$

$$v_{t_1}^{-,\text{imp}} = v_{t_1}^{-} - \frac{1}{Z_s^{-}}(\tau_1^{\text{corr}} - \tau_1^{-}) \tag{11b}$$

$$v_{t_2}^{-,\text{imp}} = v_{t_2}^{-} - \frac{1}{Z_s^{-}}(\tau_2^{\text{corr}} - \tau_2^{-}) \tag{11c}$$

Plus side ($\mathbf{Q}^{+}$):

$$v_n^{+,\text{imp}} = v_n^{+} + \frac{1}{Z_p^{+}}(\sigma_n^{\text{corr}} - \sigma_n^{+}) \tag{12a}$$

$$v_{t_1}^{+,\text{imp}} = v_{t_1}^{+} + \frac{1}{Z_s^{+}}\left(\tau_1^{\text{corr}} - \tau_1^{+}\right) \tag{12b}$$

$$v_{t_2}^{+,\mathrm{imp}} = v_{t_2}^{+} + \frac{1}{Z_s^{+}}\left(\tau_2^{\mathrm{corr}} - \tau_2^{+}\right) \tag{12c}$$

Both sides (stress):

$$\sigma_n^{\pm,\mathrm{imp}} = \sigma_n^{\mathrm{corr}}, \quad \tau_1^{\pm,\mathrm{imp}} = \tau_1^{\mathrm{corr}}, \quad \tau_2^{\pm,\mathrm{imp}} = \tau_2^{\mathrm{corr}} \tag{11d/12d}$$

Code-to-math dictionary for Eq. (11)-(12):

Math	Code	Meaning
$v_{t_1}^{+,\mathrm{imp}}$	`imposed_plus[VY]`	Imposed tangent-1 velocity, plus side
$1/Z_s^{+}$	`invZs_plus = 1.0 / data.Zs_plus`	Inverse S-impedance, plus side
τ_1^{corr}	`t1_corr`	Corrected tangent-1 traction
τ_1^{+}	`Q_plus[SXY]`	Current tangent-1 traction, plus side

Verification: Subtracting (12b) from (11b): $v_{t_1}^{+,\mathrm{imp}} - v_{t_1}^{-,\mathrm{imp}} = V_1$ (slip rate). This confirms the velocity jump equals the slip rate, as required.

2.4.5 4.5 Implementation: Fault-Local to Global Rotation (R-005 Fix)

(R-005) Eqs. (7)–(12) are derived in **fault-local** coordinates (normal n, tangent t_1, tangent t_2). The code dictionary above maps to fault-local indices (0–8 in the rotated frame), **not** global enums like `VY`/`SXY`. For a general fault orientation (e.g., BP5 fault at $Y = 0$ where the normal is $(0, 1, 0)$, not $(1, 0, 0)$), the implementation must rotate states before and after the Riemann solve:

4-step rotation pipeline:

1. **Global → fault-local:** Rotate $\mathbf{Q}^{\pm}$ from global coordinates to fault-local using `FaultBasis`:
   - $\mathbf{Q}^{\pm}_{\mathrm{local}} = \mathbf{T}^{-1}\,\mathbf{Q}^{\pm}_{\mathrm{global}}$
   - Code: `FaultBasis::ProjectTraction()` handles the stress rotation; velocity rotation via $\mathbf{T}_v^{-1}$
2. **Trial traction + friction solve** (Eqs. 7–10): All in fault-local frame. The indices in the code dictionary (e.g., σ_n = `Q_local[0]`, τ_1 = `Q_local[3]`, v_n = `Q_local[6]`) refer to **rotated** components.
3. **Construct imposed state** (Eqs. 11–12): In fault-local frame, producing $\mathbf{Q}^{\pm,\mathrm{imp}}_{\mathrm{local}}$.
4. **Fault-local → global:** Rotate imposed states back:
   - $\mathbf{Q}^{\pm,\mathrm{imp}}_{\mathrm{global}} = \mathbf{T}\,\mathbf{Q}^{\pm,\mathrm{imp}}_{\mathrm{local}}$
   - Code: `FaultBasis::EmbedSlip()` handles the inverse rotation (fault-local to global)

How the DG method uses the imposed state: In the face flux loop, instead of the standard Godunov flux (Eq. 4), the fault face applies:

$$\mathbf{F}_n^{h,\mathrm{fault+}} = \mathbf{A}_n^{+}\,\mathbf{Q}^{+,\mathrm{imp}}, \qquad \mathbf{F}_n^{h,\mathrm{fault-}} = \mathbf{A}_n^{-}\,\mathbf{Q}^{-,\mathrm{imp}}$$

Each element receives the flux computed from its imposed state — this ensures the DG scheme "sees" a consistent friction-limited interface.

2.4.6 4.5 State Variable Update

$$\boxed{\psi(t+\Delta t) = \psi(t)\, e^{-V\Delta t/L} + \frac{L}{V}\left(1 - e^{-V\Delta t/L}\right)} \tag{13}$$

Code: `UpdateStateAnalytic(psi_old, V, Dc, dt)` using `expm1()` for numerical stability.

2.4.7 4.6 CFL Condition

$$\boxed{\Delta t \le \mathrm{CFL}\,\frac{h_{\min}}{c_p}, \qquad \mathrm{CFL} \sim \frac{1}{2N+1}} \tag{14}$$

where $h_{\min}$ is the minimum inscribed diameter and N is the polynomial order.

2.4.8 4.7 Dual Friction Solver: Brent vs Newton-Raphson

Both methods solve the same Eq. (8). The choice depends on CPU vs GPU execution.

Brent's method (SEAS-MFEM default for CPU):

- Bracket: $[V_{\mathrm{lo}} = 0,\ V_{\mathrm{hi}} = \Theta/\eta_s]$. **(R-002 fix)** Guaranteed sign change because $g(0) = 0 + 0 - \Theta = -\Theta < 0$ and $g(\Theta/\eta_s) = |\sigma_n|\, f(\Theta/\eta_s, \psi) + \Theta - \Theta = +|\sigma_n|\, f > 0$. Note: this is the **opposite polarity** from the QD Brent solver where $F(V_{\mathrm{lo}}) > 0$ — ensure the Brent implementation handles both orderings.
- Convergence: superlinear (~ 1.62 order), typically **8–15 iterations**
- **Never fails** for any parameter range (guaranteed by bracketing)
- Robust for extreme $\psi/a > 100$ where true $V \sim 10^{-86}$
- No derivative required

Newton-Raphson (default for GPU, from SeisSol):

$$g(\hat{V}) = |\sigma_n|\, f(\hat{V}, \psi) + \eta_s\, \hat{V} - \Theta = 0$$

$$\frac{dg}{d\hat{V}} = |\sigma_n|\, \frac{df}{d\hat{V}} + \eta_s$$

$$\frac{df}{d\hat{V}} = \frac{a\, C}{\sqrt{1 + (\hat{V}\, C)^2}}, \qquad C = \frac{1}{2V_0} \exp\!\left(\frac{\psi}{a}\right) \tag{15}$$

$$\hat{V}_{i+1} = \max\!\left(10^{-45},\ \hat{V}_i - \frac{g}{dg/d\hat{V}}\right)$$

- Convergence: quadratic, typically **2–5 iterations** for normal parameters
- **Can fail** for extreme $\psi/a > 200$ (iteration stuck at floor 10^{-45})
- Requires derivative (Eq. 15), but derivative is cheap

Why Newton is better on GPU:

Property	Brent	Newton-Raphson
Avg iterations	8–15	2–5
Max iterations	~20	~60 (with floor guard)
Thread divergence	High (conditional interpolation vs bisection)	Low (uniform code path)
Warp efficiency	~90%	~95%

Property	Brent	Newton-Raphson
Derivative cost	None	~20% overhead per iteration
Total GPU cost	~15 iterations × 1.0	~4 iterations × 1.2 = **~5 units**

GPU prefers fewer iterations because all threads in a warp must wait for the slowest thread. Newton's 2–5 iterations with uniform code path is much better than Brent's 8–15 iterations with branch-heavy logic.

Implementation strategy:

```
class FrictionSolver {
public:
   enum class Method { Brent, NewtonRaphson, HybridNRBisection };

   // CPU default: Brent (robust for all parameter ranges)
   // GPU default: NewtonRaphson (fewer iterations, less divergence)
   static Method DefaultMethod() {
      return Device::IsEnabled() ? Method::NewtonRaphson : Method::Brent;
   }

   real_t Solve(real_t tau, real_t psi, real_t sigma_n,
                real_t eta, real_t a, Method method) const;
};
```

Hybrid NR+Bisection (third option, safest for GPU):

1. Try Newton for 5 iterations
2. If not converged, fall back to bisection with bracket $[0, \Theta/\eta_s]$
3. Bisection converges in $\sim$ 20 more iterations (all threads in lockstep)
4. Total worst-case: 25 iterations, guaranteed convergence

---

3 PART II: IMPLEMENTATION PHASES

3.1 Phase 1: WaveOperator Core — Volume Integrals + Interior Flux

3.1.1 Goal

A `WaveOperator<ParMesh>` that inherits only `TimeDependentOperator` (R-001), solves the velocity-stress wave equation on a homogeneous cube with Godunov flux at interior faces, verified by plane wave convergence.

3.1.2 Files to Create

- `dynamic/wave_state.hpp` — **Q** index enum (`QIndex`), energy density utilities
- `dynamic/wave_operator.hpp` — `WaveOperator<MeshType>` class (see class definition in Overview above)
- `dynamic/wave_operator.cpp` — `Mult()`, `ComputeVolumeRHS()`, `ComputeFaceFluxRHS()`, `AssembleElementMassInverse()`
- `dynamic/godunov_flux.hpp` — `GodunovFlux` class: Jacobians $\mathbf{A}$,$\mathbf{B}$,$\mathbf{C}$; rotation $\mathbf{T}$,$\mathbf{T}^{-1}$; split flux $\mathbf{A}^{\pm}$; `Interior()`, `Absorbing()`, `FreeSurface()` methods
- `dynamic/godunov_flux.cpp` — Implementation (~400 LOC)
- `dynamic/seas_dynamic_operator.hpp` — `SEASDynamicOperator<MeshType>` coupling class (R-001)
- `tests/unit/test_godunov_flux.cpp` — 8 flux unit tests
- `tests/unit/test_wave_operator.cpp` — 6 operator integration tests

3.1.3 Files to Modify

- `domain/boundary_config.hpp` — Add `absorbing_attrs` field (one line)
- `domain/domain_config.hpp` — Add `cfl_factor` field (one line)
- `Makefile` — Add dynamic rupture build targets

3.1.4 Detailed Requirements

1. **`GodunovFlux` construction:** From λ, μ, ρ, compute c_p, c_s, and precompute split flux matrices $\mathbf{A}_x^+, \mathbf{A}_x^-$ (9$×$9 each). These are constant for homogeneous material.
2. **`GodunovFlux::Interior()`:** Implements Eq. (4) via: rotate → apply split → rotate back. Reference: SeisSol `ElasticSetup.h:146–165`.
3. **`WaveOperator::Mult()`:** Implements Eq. (3): `dQdt = 0; ComputeVolumeRHS(Q, dQdt); ComputeFaceFluxRHS(Q, dQdt); ApplyMassInverse(dQdt);`
4. **`ComputeVolumeRHS()`:** Implements Eq. (2c). For each element e, quadrature point q: evaluate $\mathbf{Q}_h(\mathbf{x}_q)$ by interpolation, compute fluxes $\mathbf{A}_j\mathbf{Q}$ for $j = x, y, z$, accumulate `rhs_e[c*ndof+i] += w * dshape(i,j) * F[j][c]`.
5. **`ComputeFaceFluxRHS()`:** Implements Eq. (2d). For each face f: evaluate $\mathbf{Q}$ on both sides, compute numerical flux via `GodunovFlux::Interior/Absorbing/FreeSurface`, accumulate into both elements' RHS.
6. **`AssembleElementMassInverse()`:** For each element, assemble $M_{kl}^{(e)} = \int_{T_e} \Phi_k \Phi_l \, dV$ and invert (dense $n_{\text{dof}} \times n_{\text{dof}}$ per element). Store as `elem_mass_inv_[e]`.
7. **`SEASDynamicOperator`:** Wraps `WaveOperator*` + `FaultFaceFlux*`. Its `Mult()` calls `wave_->Mult(Q, dQdt)` (which internally dispatches fault faces to `FaultFaceFlux` if set).

3.1.5 Unit Tests

#	Test Name	File	What	Input	Expected	Tolerance
1	`TestJacobianSymmetry`	`test_godunov_flux.cpp`	$\mathbf{A}_x, \mathbf{B}_y, \mathbf{C}_z$ nonzero entries correct	$\lambda = 32.04 \times 10^9$, $\mu = 32.04 \times 10^9$, $\rho = 2670$	$A_{6,0} = -1/\rho$, $A_{0,6} = -(\lambda + 2\mu)$	Exact
2	`TestEigenvalues`	`test_godunov_flux.cpp`	Eigenvalues of $\mathbf{A}_n$	5 random unit normals	$\{\pm c_p, \pm c_s, \pm c_s, 0, 0, 0\}$	$< 10^{-10}$
3	`TestRotationOrthogonality`	`test_godunov_flux.cpp`	$\mathbf{T}\mathbf{T}^{-1} = \mathbf{I}$	$\mathbf{n} = (1,0,0)$, $(0,1,0)$, $(0,0,1)$, $(1,1,1)/\sqrt{3}$	Identity	$\|\cdot\|_F < 10^{-14}$
4	`TestInteriorFluxConsistency`	`test_godunov_flux.cpp`	$\mathbf{Q}_L = \mathbf{Q}_R \Rightarrow \mathbf{F} = \mathbf{A}_n\mathbf{Q}$	Random uniform $\mathbf{Q}$	No dissipation	$< 10^{-12}$
5	`TestInteriorFluxConservation`	`test_godunov_flux.cpp`	$\mathbf{F}(\mathbf{Q}_L, \mathbf{Q}_R, \mathbf{n}) + \mathbf{F}(\mathbf{Q}_R, \mathbf{Q}_L, -\mathbf{n}) = 0$	Random $\mathbf{Q}_L$, $\mathbf{Q}_R$	Zero sum	$< 10^{-12}$
6	`TestAbsorbingOutgoingOnly`	`test_godunov_flux.cpp`	Outgoing P-wave: full flux; incoming: zero	Analytical P-wave states	Outgoing = $\mathbf{A}\mathbf{Q}$, incoming = 0	$< 10^{-12}$

#	Test Name	File	What	Input	Expected	Tolerance
7	`TestFreeSurfaceZeroTraction`	`test_godunov_flux.cpp`	Γ mirror $\Rightarrow \sigma \cdot \mathbf{n} = 0$	Random $\mathbf{Q}$	Zero normal traction	$< 10^{-10}$
8	`TestSplitFluxConsistency`	`test_godunov_flux.cpp`	$\mathbf{A}^+ + \mathbf{A}^- = \mathbf{A}_x$	Compare matrices	Equality	$< 10^{-14}$
9	`TestMassMatrixInverse`	`test_wave_operator.cpp`	$\mathbf{M}\mathbf{M}^{-1} = \mathbf{I}$ per element	2×2×2 hex mesh, order 2	Identity	$< 10^{-12}$
10	`TestPlaneWavePSpeed`	`test_wave_operator.cpp`	P-wave propagates at c_p	P-wave IC, advance 1 period	Phase error < 1% at 10 elem/wavelength	$\lvert c/c_p - 1\rvert < 0.01$
11	`TestPlaneWaveSSpeed`	`test_wave_operator.cpp`	S-wave propagates at c_s	S-wave IC, advance 1 period	Phase error < 1%	$\lvert c/c_s - 1\rvert < 0.01$
12	`TestConvergenceOrder`	`test_wave_operator.cpp`	L^2 error $\propto h^{N+1}$	$h = 1/4, 1/8, 1/16$; $N = 1, 2$	Convergence rate	Rate $\geq N + 0.5$
13	`TestEnergyConservation`	`test_wave_operator.cpp`	Energy conserved in reflecting box	4×4×4, all free surface, 1000 RK4 steps	$\lvert E_f/E_0 - 1\rvert$ small	$< 10^{-10}$
14	`TestFaultInfoSetup`	`test_wave_operator.cpp`	Fault faces detected and `FaultBasis` constructed	Mesh with fault attr=3	`GetFaultBasis()` non-null, `GetNumFaultDOFs()` `> 0`	No throw

3.1.6 Acceptance Criteria

- ☐ All 14 unit tests pass
- ☐ `WaveOperator` compiles, inherits only `TimeDependentOperator` (no `DomainOperator`)
- ☐ `SEASDynamicOperator` compiles and calls `WaveOperator::Mult()`
- ☐ `make test` passes (all existing QD tests unaffected)

3.1.7 Dependencies

- Depends on: `ConstitutiveModel` (existing), `FaultBasis` (existing), `BoundaryConfig` (existing, extend)
- Required by: Phase 2, 3

3.2 Phase 2: Boundary Conditions — Three Methods

3.2.1 Goal

Implement three boundary treatment methods of increasing accuracy: - **Phase 2a: First-order ABC** (Eq. 5) — sufficient for short TPV102 - **Phase 2b: PML** — exponential absorption, good for medium-length simulations - **Phase 2c: FEM-SBI** — exact (zero reflection), **DEFERRED** (design retained for future LVZ work)

3.2.2 Phase 2a: First-Order Absorbing BC + Free Surface

Files to modify: `dynamic/godunov_flux.cpp` — implement `Absorbing()` (Eq. 5) and `FreeSurface()` (Eq. 6).

Files to create: `tests/unit/test_wave_bc.cpp` — 7 BC tests.

Absorbing (Eq. 5): Rotate $\mathbf{Q}_{\text{self}}$ to face-normal frame, apply $\mathbf{A}_x^+$ (outgoing only), rotate back. Reference: SeisSol `ElasticSetup.h:146–165` with `qGodNeighbor = 0`.

Free Surface (Eq. 6): Rotate, create ghost via Γ mirror (flip components with odd normal indices), apply standard interior flux with ghost, rotate back. Reference: SeisSol `Model/Common.h:251–296`.

Unit tests:

#	Test	What	Tolerance
15	`TestAbsorbingNormalIncidence`	P-wave exits with < 1% reflection	$E_{\text{res}}/E_{\text{in}} < 0.01$
16	`TestAbsorbingObliqueIncidence`	30° P-wave: documents reflection (regression test)	$1\% < R < 20\%$
17	`TestFreeSurfacePReflection`	$R_{PP} = -1$ at normal incidence	$\lvert R_{PP} + 1\rvert < 0.05$
18	`TestFreeSurfaceZeroTraction`	$\sigma \cdot \mathbf{n} = 0$ at surface all times	$\max \lvert\sigma \cdot \mathbf{n}\rvert < 10^{-10}$
19	`TestFreeSurfaceEnergyConservation`	All-free-surface box: energy conserved	$\lvert E_f/E_0 - 1\rvert < 10^{-10}$
20	`TestMixedBCCorner`	Corner element (free + absorbing) stable	No NaN; energy decreasing
21	`TestAbsorbingEnergyDecay`	All-absorbing box: energy monotonically decreasing	$E(t + \Delta t) \leq E(t) + \epsilon$

3.2.3 Acceptance Criteria (Phase 2a)

- ☐ All 7 BC tests pass (Tests 15–21)
- ☐ `GodunovFlux::Absorbing()` and `FreeSurface()` implemented and callable
- ☐ P-wave at normal incidence exits absorbing boundary with < 1% reflected energy
- ☐ Free-surface traction $\sigma \cdot \mathbf{n} = 0$ verified at every monitored time step
- ☐ Energy conserved in all-free-surface box to $< 10^{-10}$ relative error
- ☐ Phase 1 tests still pass
- ☐ `make test` passes (all existing QD tests unaffected)

3.2.4 Dependencies (Phase 2a)

- Depends on: Phase 1
- Required by: Phase 4 (TPV102 needs free surface + absorbing)

3.2.5 Phase 2b: Perfectly Matched Layer (PML)

Files to Create:

- `dynamic/pml_layer.hpp` — PML damping coefficient and modified equations
- `dynamic/pml_layer.cpp` — Implementation (~300 LOC)
- `tests/unit/test_pml.cpp` — 4 PML tests

Mathematical formulation: Convolutional PML (CPML) from Komatitsch & Martin (2007). In the PML region, modify Eq. (1) to:

$$\frac{\partial \mathbf{Q}}{\partial t} + \mathbf{A}\frac{\partial \mathbf{Q}}{\partial x} + \mathbf{B}\frac{\partial \mathbf{Q}}{\partial y} + \mathbf{C}\frac{\partial \mathbf{Q}}{\partial z} = -d(\mathbf{x})\,\mathbf{D}\,\mathbf{Q} \tag{16}$$

where $d(\mathbf{x})$ is a spatially varying damping coefficient and $\mathbf{D}$ is a diagonal damping matrix.

Damping profile (cubic, matching farms_aftershock `PMLCoefficientMaterial`):

$$d(\mathbf{x}) = d_{\max}\left(\frac{\text{dist}(\mathbf{x},\, \partial\Omega_{\text{comp}})}{L_{\text{PML}}}\right)^3 \tag{17}$$

where:

- $d_{\max} = \frac{3\,c_p}{2\,L_{\text{PML}}} \ln\left(\frac{1}{R_0}\right)$ with target reflection $R_0 = 10^{-3}$
- L_{PML} = PML layer thickness (5–10 km for BP5)
- $\text{dist}(\mathbf{x}, \partial\Omega_{\text{comp}})$ = distance from point to computational domain boundary

Code: `pml.ComputeDamping(x, y, z)` returns $d(\mathbf{x})$.

Damping matrix D: For a PML absorbing in the x-direction:

$$\mathbf{D}_x = \text{diag}(1,\, 0,\, 0,\, 1,\, 0,\, 1,\, 1,\, 0,\, 0)$$

Components with an x-index are damped; others are not. For corners where two PML regions overlap, $d(\mathbf{x})\,\mathbf{D} = d_x(\mathbf{x})\,\mathbf{D}_x + d_y(\mathbf{x})\,\mathbf{D}_y + d_z(\mathbf{x})\,\mathbf{D}_z$.

Implementation in `Mult()`:

(R-004 fix) At corners where PML regions overlap, each state component needs its own damping value. `ComputeDamping()` returns three directional values (d_x, d_y, d_z). The per-component damping is $d_c = d_x\,D_x[c] + d_y\,D_y[c] + d_z\,D_z[c]$, where D_x, D_y, D_z encode which components have an x-, y-, or z-index respectively.

```
// After standard volume + face assembly:
if (pml_layer_) {
   // Which components are damped by which direction:
   //                     SXX SYY SZZ SXY SYZ SXZ VX VY VZ
   static const int Dx[] = {1,  0,  0,  1,  0,  1,  1, 0, 0};
   static const int Dy[] = {0,  1,  0,  1,  1,  0,  0, 1, 0};
   static const int Dz[] = {0,  0,  1,  0,  1,  1,  0, 0, 1};

   for (int e = 0; e < ne; e++) {
      for (int q = 0; q < nqp; q++) {
         real_t dx, dy, dz;
         pml_layer_->ComputeDamping(x_q, dx, dy, dz);  // 3 directional values
         for (int c = 0; c < 9; c++) {
            real_t d_c = dx*Dx[c] + dy*Dy[c] + dz*Dz[c];
            if (d_c > 0.0)
               for (int i = 0; i < ndof; i++)
                  rhs_e[c*ndof+i] -= w * d_c * shape(i) * Q_qp[c];
         }
      }
   }
}
```

PML unit tests:

#	Test	What	Tolerance
15b	`TestPMLNormalReflection`	P-wave exits with $< 0.1\%$ reflection	$E_{\text{res}}/E_{\text{in}} < 10^{-3}$
16b	`TestPMLObliqueReflection`	45° P-wave: $< 1\%$ reflection (vs 20% for ABC)	$E_{\text{res}}/E_{\text{in}} < 0.01$
17b	`TestPMLEnergyDecay`	Energy monotonically decreasing, faster than ABC	$E_{\text{PML}}(t) < E_{\text{ABC}}(t)$
18b	`TestPMLCorner`	Two overlapping PML layers at corner are stable	No NaN after 500 steps

3.2.6 Acceptance Criteria (Phase 2b)

- ☐ All 4 PML tests pass (Tests 15b–18b)
- ☐ `PMLLayer` class compiles with directional damping `ComputeDamping(x, dx, dy, dz)` (R-004)
- ☐ PML reflection at normal incidence $< 0.1\%$ (10× better than first-order ABC)
- ☐ PML reflection at 45° oblique $< 1\%$ (20× better than first-order ABC)
- ☐ Per-component damping at corners produces no instability for 500 steps
- ☐ Phase 1 and 2a tests still pass

3.2.7 Dependencies (Phase 2b)

- Depends on: Phase 1 (WaveOperator with PML hook)
- Required by: Phase 5 (BP5 hybrid uses PML for boundary absorption with Phase 2c deferred)

3.2.8 Phase 2c: Hybrid FEM-SBI Boundary — DEFERRED

> **Status: DEFERRED.** The FEM-SBI design below is retained as a long-term architecture reference but is **not part of the current implementation scope**. Phases 1, 2a, 2b, 3, 4, 5, 6 proceed without Phase 2c. When FEM-SBI is needed (e.g., for LVZ studies), revisit this section and promote it to an active phase.
>
> **For the current BP5 hybrid work, PML (Phase 2b) provides sufficient boundary absorption.** SBI will be implemented later when heterogeneous near-fault structure is required.

Rationale: For long-term BP5 hybrid simulations with heterogeneous near-fault structure (low-velocity zones, damage rheology), pure SBI is insufficient because it assumes a homogeneous elastic half-space everywhere. The **Hybrid FEM-SBI** approach (Abdelmeguid et al. 2019, *JGR Solid Earth*, 10.1029/2019JB018036) confines the FEM domain to a near-fault strip containing heterogeneities, while using SBI to represent the exact half-space response at virtual boundaries. This eliminates boundary reflections (SBI is exact) while retaining FEM's ability to model complex near-fault physics.

The Abdelmeguid et al. paper uses CG FEM. No published work couples DG with SBI for earthquake cycles. Our adaptation from CG to DG is novel. The key reference for DG-BEM coupling theory is Of, Rodin, Steinbach & Taus (2012, *SIAM J. Numer. Anal.*, 10.1137/110848530).

Files to Create:

- `dynamic/sbi_kernel.hpp` — FFT-based traction computation (DtN map)
- `dynamic/sbi_kernel.cpp` — Implementation (~300 LOC, R-003: 2D FFT)
- `dynamic/sbi_boundary.hpp` — SBI boundary condition for WaveOperator (QD Neumann + DR ghost state)
- `dynamic/sbi_boundary.cpp` — Implementation (~400 LOC)
- `tests/unit/test_sbi_boundary.cpp` — 5 SBI tests

Dependency: FFTW3 (optional; compile-time flag `SEAS_USE_FFTW`).

3.2.8.1 2c.1 Domain Decomposition (Abdelmeguid et al. 2019, Fig. 1 and Eq. 9) The computational domain is decomposed into:

- **FEM strip** $\Omega_{\text{FEM}} = [-W_s, +W_s] \times [0, L_z]$: near-fault region modeled by DG. Contains the fault, any LVZ, damage zones, or rheological complexity. Width W_s is chosen to enclose all heterogeneities (e.g., W_s = 1–5 km for BP5).
- **Two half-spaces** S^+ (right, $x > W_s$) and S^- (left, $x < -W_s$): homogeneous elastic, modeled exactly by SBI. No mesh needed in these regions.
- **Virtual boundaries** S^+_{SBI} at $x = +W_s$ and S^-_{SBI} at $x = -W_s$: where FEM and SBI exchange traction and displacement.

The paper's governing equation for the FEM strip (Eq. 9):

$$-\int_V \sigma_{ij}\,\phi_{i,j}\,dV + \int_{S^+_{\text{SBI}}} \tau_i^{+,\text{SBI}}\,\phi_i\,dS - \int_{S^-_{\text{SBI}}} \tau_i^{-,\text{SBI}}\,\phi_i\,dS - \int_V \rho\,\ddot{u}_i\,\phi_i\,dV - \int_{S_{f+}} T_i^{f+}\phi_i\,dS + \int_{S_{f-}} T_i^{f-}\phi_i\,dS = 0$$

where $\tau^{\pm,\text{SBI}}$ are the SBI tractions at virtual boundaries, and $T^{f\pm}$ are the fault tractions (Lagrange multipliers).

In compact matrix form (Eq. 16-17):

$$\mathbf{K}\mathbf{u}(t) + \mathbf{L}^T\big(\tau^{\text{SBI}}(t) + \mathbf{T}^f(t)\big) = \mathbf{F}(t)$$

$$\mathbf{L}\mathbf{u}(t) = \mathbf{D}(t) \quad \text{(slip constraint on fault)}$$

3.2.8.2 2c.2 SBI DtN Kernel (Abdelmeguid et al. 2019, Eq. 18-22) The SBI traction at a virtual boundary is computed via the Dirichlet-to-Neumann map. For the antiplane case (paper's Eq. 18, 22), the traction at virtual boundary S_{SBI} is:

$$\tau^{\text{SBI}}(x_1, t) = \tau^0(x_1, t) \mp \frac{\mu}{c_s}\dot{u}_{\text{SBI}}(x_1, t) \pm f^*(x_1, t) \qquad \text{(18-paper)}$$

where: - τ^0 = locked-fault traction (stress without slip) - $\frac{\mu}{c_s}\dot{u}_{\text{SBI}}$ = impedance term (radiation from the virtual boundary) - f^* = space-time convolution of the Green's function with the boundary displacement history

In the quasi-dynamic limit, the convolution f^* is replaced by the static kernel (paper's Eq. 22):

$$F_s^{\pm}(t; q) = \mp\mu|q_s|\,U_s^{\pm}(t; q) \qquad \text{(22-paper)}$$

where $q_s = 2\pi s/\lambda$ is the wavenumber and U_s is the Fourier coefficient of boundary displacement. In real space:

$$f^{\pm}(z, t) = \mp\mu \cdot \mathcal{F}^{-1}[|q|\;\hat{u}_{\text{SBI}}(z, t)]$$

For the 3D case (R-003 fix), this becomes a 2D Fourier transform:

$$\boxed{f^{\pm}(y, z, t) = \mp\mu \cdot \mathcal{F}_{2D}^{-1}[|k|\;\hat{u}_{\text{SBI}}(y, z, t)]\,, \qquad |k| = \sqrt{k_y^2 + k_z^2}} \tag{19}$$

Critical difference from the fault SBI kernel: The DtN kernel at virtual boundaries uses $\mu|k|$ (no factor of π), while the fault SBI kernel uses $\pi\mu|k|$. The factor of π arises from the fault dislocation involving both sides of the half-space (see existing plan `sbionly_implementation_plan_02282026.md`).

3.2.8.3 2c.3 Predictor-Corrector Algorithm (Abdelmeguid et al. 2019, Algorithm 1)

The paper's time-stepping algorithm, adapted for our DG solver:

At each RK45 stage, given slip $d(t)$, state $\theta(t)$, displacement $u(t)$ and $u(t-\Delta t)$:

1. **Predict** $u^*_{\text{SBI}}(t) = u_{\text{SBI}}(t-\Delta t)$ (extrapolate from previous step)
2. Compute SBI traction prediction: $\tau^{\text{SBI},*}(t) = \mp\frac{\mu}{c_s}\dot{u}^*_{\text{SBI}} + f^*(t)$
3. **Solve** DG system with predicted SBI traction as Neumann BC → get $u^{**}(t)$
4. **Correct** boundary displacement: $u_{\text{SBI}}(t) = \frac{1}{2}[u^*_{\text{SBI}}(t) + u^{**}_{\text{SBI}}(t)]$
5. Re-solve with corrected SBI traction (optional, for improved accuracy)
6. Extract fault traction $T^f(t)$ and solve friction law: $T^f = F(V,\theta)\,\sigma_n + \eta V$
7. Advance state variables via RK45

For quasi-dynamic, the paper reports that a single correction step (steps 1-4) is sufficient for convergence.

3.2.8.4 2c.4 Adapting CG-SBI Coupling to DG

No published work exists for DG-SBI earthquake coupling. Our adaptation is based on:

1. **Of et al. (2012)** — DG-BEM coupling theory for Laplace/Helmholtz, proves stability for IPDG
2. **Betcke et al. (2022)** — FEM-BEM via Nitsche's method, related to DG penalty terms

How SBI traction enters the DG formulation:

For quasi-static/quasi-dynamic (SIPG/BR2 bilinear form): The SBI traction enters as a **pure Neumann BC** through the linear form only:

$$L^{\text{SBI}}(v_h) = \sum_{e\in\mathcal{F}_{\text{SBI}}} \int_e \tau^{\text{SBI}}\, v_h\, dS$$

This is implemented via `BoundaryLFIntegrator` in MFEM. The bilinear form (stiffness matrix) has **no boundary face terms** for Neumann faces — no penalty, no consistency, no symmetry term. The stiffness matrix $\mathbf{K}$ is unchanged from the standard DG assembly.

In DG, each boundary face has only one state (the interior trace from the adjacent element). There is no "ghost" element. The SBI traction replaces the need for a ghost state entirely.

For fully-dynamic (Godunov flux): The SBI provides a ghost state for the upwind flux:

$$\mathbf{F}^{\text{SBI}}_n = \mathbf{A}^+_n\,\mathbf{Q}_{\text{self}} + \mathbf{A}^-_n\,\mathbf{Q}_{\text{SBI}}$$

where $\mathbf{Q}_{\text{SBI}}$ is constructed from: - Stress components: from the DtN traction (Eq. 19) - Velocity components: extrapolated from the previous time step

Projection pipeline (DG boundary ↔ SBI uniform grid):

1. **DG → SBI grid:** Evaluate u_h at uniform SBI grid points along the virtual boundary using `GridFunction::GetValues()`. Since DG elements tile the boundary without overlap, each face contributes values within its footprint.
2. **Apply DtN:** FFT on uniform grid, multiply by $\mu|k|$, inverse FFT. The DtN is a *global* operator — it must see the entire boundary at once.
3. **SBI grid → DG faces:** Interpolate the resulting SBI traction from the uniform grid to DG face quadrature points via `SBIDtNTractionCoefficient::Eval(x_qp)`.

Stability: The coupling is energy-stable because the SBI provides the physically correct half-space traction (energy-neutral boundary). DG numerical dissipation (from interior penalty terms) acts only inside the domain. The explicit/lagged coupling is controlled by the RK45 error estimator (Abdelmeguid et al. 2019, Algorithm 1).

3.2.8.5 2c.5 Why FEM-SBI over Pure SBI for Long-Term Plan

Feature	Pure SBI	FEM-SBI (Hybrid)
Homogeneous half-space	Exact	Exact (via SBI far-field)
Low-velocity fault zones	Cannot model	FEM strip captures LVZ
Damage/plasticity near fault	Cannot model	FEM handles nonlinear rheology
Heterogeneous material	Cannot model	FEM strip can have variable $\mu(x)$
Computational cost	$O(N \log N)$ per step	$O(N_{\text{FEM}}^{1.5}) + O(N_{\text{SBI}} \log N_{\text{SBI}})$
Boundary reflections	Zero	Zero (SBI at virtual boundaries)
Mesh requirement	None (1D fault grid)	Small FEM strip only

For BP5 (homogeneous), pure SBI suffices. For future problems with LVZs (Abdelmeguid et al. 2019, Fig. 2b-c), the FEM strip must be wide enough to contain all heterogeneity.

Implementation phasing: - Phase 2c-i: Pure SBI DtN at virtual boundaries (homogeneous, quick win for BP5) - Phase 2c-ii: Full FEM-SBI coupling with predictor-corrector (for LVZ studies)

3.2.8.6 2c.6 SBI Unit Tests

#	Test	What	Tolerance
15c	`TestSBIKernelDC`	Constant displacement → zero traction ($\|k\| = 0$ mode)	$< 10^{-14}$
16c	`TestSBIKernelSingleMode`	Single Fourier mode $k = 1$: traction amplitude $= \mu\|k\| \hat{u}$	$< 10^{-12}$
17c	`TestSBIZeroReflection`	P-wave exits SBI boundary with **zero** reflection	$E_{\text{res}}/E_{\text{in}} < 10^{-10}$
18c	`TestSBIObliqueZeroReflection`	45° oblique: still zero	$E_{\text{res}}/E_{\text{in}} < 10^{-10}$
19c	`TestSBILongTimeStability`	10,000 steps: no energy growth	$E(t) \leq E(0)$ for all t

3.2.9 Acceptance Criteria (Phase 2c) — DEFERRED

These criteria apply when Phase 2c is promoted to active implementation.

- ☐ All 5 SBI tests pass (Tests 15c–19c)
- ☐ `SBIKernel` class compiles with 2D FFTW r2c plans (R-003)
- ☐ DC component (constant displacement) produces zero traction
- ☐ P-wave exits SBI boundary with $< 10^{-10}$ reflected energy (effectively zero)
- ☐ No energy growth over 10,000 time steps (long-time stability)
- ☐ DG ↔ SBI projection pipeline (evaluate at uniform grid, FFT, interpolate back) is functional
- ☐ Phase 1, 2a, 2b tests still pass
- ☐ Builds with and without `SEAS_USE_FFTW` (graceful fallback to ABC when FFTW unavailable)

3.2.10 Dependencies (Phase 2c) — DEFERRED

- Depends on: Phase 1 (WaveOperator with SBI face type), FFTW3 library
- Required by: Future BP5 hybrid with exact boundary treatment (not required by any current phase)

3.2.11 Boundary Method Selection for BP5

Scenario	Recommended BC	Why
TPV102 (12 s, short)	First-order ABC	Domain large enough; reflections don't reach fault
BP5 single event (~10 s dynamic)	PML	Low reflection at all angles; moderate complexity
BP5 full cycle (100+ yr hybrid)	SBI	**Exact** zero reflection; eliminates cumulative contamination

---

3.3 Phase 3: Fault-Face Riemann Solver

3.3.1 Goal

Dynamic rupture on a fault interface, with the Riemann solver reusing `DieterichRuinaFriction::SolveSlipRateVectorPsi()` for the friction solve (Eq. 8) and `FaultBasis` for coordinate transforms (Section 4.5 rotation pipeline).

3.3.2 Files to Create

- `dynamic/fault_face_flux.hpp` — `FaultFaceFlux` class: `DOFData` struct, `ComputeTrialTraction()`, `Evaluate()`, `UpdateStateAnalytic()` (~300 LOC)
- `dynamic/fault_face_flux.cpp` — Implementation
- `dynamic/friction_solver.hpp` — `FrictionSolver` class with Brent/NR/Hybrid methods
- `tests/unit/test_fault_face_flux.cpp` — 10 fault flux tests
- `tests/unit/test_friction_solver.cpp` — 5 dual solver tests

3.3.3 Files to Modify

- `dynamic/wave_operator.cpp` — Replace fault face stub with `FaultFaceFlux::Evaluate()` dispatch

3.3.4 Detailed Requirements

1. **`FaultFaceFlux::ComputeTrialTraction()`:** Implements Eq. (7a-c) in fault-local coordinates. Input: `DOFData` (impedances, initial stress) + `Q_plus[9]`, `Q_minus[9]` (already rotated to fault-local by caller). Output: `sigma_n_trial`, `tau1_trial`, `tau2_trial`. Reference: SeisSol `FrictionSolverCommon.h:182–192`.
2. **`FaultFaceFlux::Evaluate()`:** Full pipeline Eq. (7)→(8)→(9)→(10)→(11)-(12)→(13). Caller rotates $\mathbf{Q}^{\pm}$ to fault-local before calling, rotates imposed states back to global after (Section 4.5 pipeline). Reference: SeisSol `precomputeStressFromQInterpolated` + `postcomputeImposedStateFromNewStress`.
3. **`FrictionSolver`:** Three methods — Brent (CPU default), NR (GPU default), HybridNRBisection. Signatures:
   - `real_t SolveBrent(real_t tau, real_t psi, real_t sigma_n, real_t eta, real_t a) const`
   - `real_t SolveNR(real_t tau, real_t psi, real_t sigma_n, real_t eta, real_t a) const`
   - NR derivative: Eq. (15), $df/d\hat{V} = aC/\sqrt{1 + (\hat{V}C)^2}$. Reference: SeisSol `SlowVelocityWeakeningLaw.h:100–107`.

4. **`UpdateStateAnalytic()`:** Implements Eq. (13) using `expm1()` for stability. Reference: SeisSol `AgingLaw.h:39–49`.

3.3.5 Unit Tests

#	Test	What	Input	Expected	Tolerance
22	`TestTrialTractionLockedFault`	$\mathbf{Q}^+ = \mathbf{Q}^-$ → trial = background	Equal states, $\sigma_{xz} = 75$ MPa	Trial = existing traction	$< 10^{-10}$
23	`TestTrialTractionVelocityJump`	Pure $\Delta v_{t_1} = 1$ m/s → predictable trial	$v^+ = +0.5$, $v^- = -0.5$, stress=0	$\tau_1^{\text{trial}} = -\eta_s$	$< 10^{-12}$
24	`TestLockedFaultHighFriction`	High friction → $V \approx 0$	$a = 0.5$, $\sigma_n = 120$ MPa, small τ	$V < 10^{-20}$	$V < 10^{-20}$
25	`TestFrictionlessFullStressDrop`	Zero friction → $\hat{V} = \Theta/\eta_s$	$a = 0$, $b = 0$, $f_0 = 0$	Full stress drop	$< 10^{-10}$
26	`TestImposedStateContinuity`	Traction continuous: $\tau^{+,\text{imp}} = \tau^{-,\text{imp}}$	Random $\mathbf{Q}^{\pm}$, TPV102 params	Equal on both sides	$< 10^{-14}$
27	`TestImposedStateVelocityJump`	$v^{+,\text{imp}} - v^{-,\text{imp}} = V$	Same setup	Jump = slip rate	$< 10^{-12}$
28	`TestSolverReuse`	Same V from standalone Brent and from `Evaluate()`	TPV102 typical values	Agreement	$< 10^{-10}$
29	`TestStateVariableEvolution`	ψ after 1000 constant-V steps matches exact	$\psi_0 = 0.5$, $V = 10^{-3}$, $L = 0.02$, $\Delta t = 10^{-3}$	ψ_{exact}	$< 10^{-10}$
30	`TestEnergyBalance`	$\Delta E_{\text{domain}} + \int \tau \cdot V \, dA \, dt = 0$	2-element mesh with fault, 100 steps	Conservation	$< 10^{-8} E_0$
31	`TestFaultFluxSignConvention`	Positive V → $\tau^{\text{corr}} < \tau^{\text{trial}}$	$\tau_0 > 0$, small perturbation	Stress drop	Qualitative

Additional unit tests for dual solver:

#	Test	What	Tolerance
32	`TestNRConvergence`	NR converges in ≤ 10 iterations for typical TPV102 params	iterations ≤ 10
33	`TestNRDerivative`	Analytical derivative (Eq. 15) matches finite difference	$\lvert df_{\text{anal}} - df_{\text{fd}} \rvert / \lvert df_{\text{fd}} \rvert < 10^{-6}$

#	Test	What	Tolerance
34	`TestBrentNREquivalence`	Brent and NR give same V for 1000 random parameter sets	$\lvert V_{\text{Brent}} - V_{\text{NR}}\rvert / \lvert V_{\text{Brent}}\rvert < 10^{-8}$
35	`TestNRExtremePsi`	NR with psi/a = 200: verify convergence or graceful fallback	Returns valid V or falls back to Brent
36	`TestHybridNRBisection`	Hybrid solver converges for all 1000 random cases	100% convergence

3.3.6 Acceptance Criteria (Phase 3)

- ☐ All 10 fault flux tests pass (Tests 22–31)
- ☐ All 5 dual solver tests pass (Tests 32–36)
- ☐ `FaultFaceFlux::Evaluate()` produces continuous traction across fault ($< 10^{-14}$)
- ☐ Velocity jump across fault equals slip rate ($< 10^{-12}$)
- ☐ Brent and NR solvers agree to $< 10^{-8}$ for 1000 random parameter sets
- ☐ NR derivative (Eq. 15) matches finite-difference to $< 10^{-6}$ relative error
- ☐ State variable after 1000 constant-V steps matches analytical to $< 10^{-10}$
- ☐ Energy balance: $|\Delta E_{\text{domain}} + \int \tau \cdot V \, dA \, dt| < 10^{-8} \, E_0$
- ☐ `WaveOperator` with fault flux produces no NaN for 100 steps with TPV102 parameters
- ☐ Phase 1 and 2 tests still pass
- ☐ `make test` passes

3.3.7 Dependencies (Phase 3)

- Depends on: Phase 1, `DieterichRuinaFriction` (existing), `FaultBasis` (existing)
- Required by: Phase 4

3.4 Phase 4: TPV102 Benchmark — Local Verification + Frontera

3.4.1 Phase 4a: Local Verification Tests (Desktop/Workstation)

Goal: Verify all TPV102 components locally before submitting cluster jobs.

Files to Create:

- `config/tpv102_params.hpp`, `config/tpv102.toml` — Parameters
- `dynamic/tpv102_setup.hpp/.cpp` — Spatial params, nucleation, stations
- `drivers/tpv102_driver.cpp` — Main driver
- `tpv102/mesh/tpv102_coarse.geo` — Coarse mesh (h=2km, ~5k elements) for local testing
- `tpv102/mesh/tpv102_medium.geo` — Medium mesh (h=1km, ~40k elements)
- `tests/unit/test_tpv102_setup.cpp` — 5 setup unit tests (from v3)
- `tests/verification/test_tpv102_local.cpp` — Local integration tests

Local integration tests (run on workstation, ~10 min each):

#	Test	Mesh	Duration	What to verify
L1	`TestTPV102NucleationOnset`	Coarse (5k)	0.5 s sim / ~2 min wall	Rupture initiates at hypocenter; V_max > 0.1 m/s
L2	`TestTPV102ShortRupture`	Coarse (5k)	2.0 s sim / ~5 min wall	Rupture propagates bilaterally; no instability

#	Test	Mesh	Duration	What to verify
L3	`TestTPV102FreeSurface`	Coarse (5k)	2.0 s	Surface displacement shows P-wave arrival
L4	`TestTPV102AbsorbingBC`	Coarse (5k)	2.0 s	No visible boundary reflections
L5	`TestTPV102StationOutput`	Coarse (5k)	2.0 s	Station files written; columns correct
L6	`TestTPV102ConvergenceCoarseVsMedium`	Both	10 s	Slip rate at hypocenter: coarse vs medium within 20%

All local tests must pass before submitting Frontera jobs.

3.4.2 Acceptance Criteria (Phase 4a)

- ☐ All 5 setup unit tests pass (boxcar, spatial a, equilibrium, nucleation, stations)
- ☐ All 6 local integration tests pass (L1–L6)
- ☐ Rupture nucleates at hypocenter within $t < 0.5$ s on coarse mesh
- ☐ Rupture propagates bilaterally without instability for 2 s
- ☐ Station output files are written with correct format (time, slip, V, traction, $\log_{10}\theta$)
- ☐ Coarse-to-medium convergence: slip rate at hypocenter within 20%
- ☐ Phase 1, 2, 3 tests still pass
- ☐ `make test` passes

3.4.3 Dependencies (Phase 4a)

- Depends on: Phase 1, 2a, 3
- Required by: Phase 4b

3.4.4 Phase 4b: Frontera Full Benchmark

Goal: Full 12 s TPV102 on production mesh, compared against SeisSol and community results.

Files to Create:

- `tpv102/mesh/tpv102_fine.geo` — Fine mesh (h=500m, ~300k elements, matching SeisSol)
- `tpv102/scripts/frontera_tpv102.sbatch` — SLURM job script
- `tpv102/scripts/compare_seissol.py` — Python comparison tool
- `tpv102/reference/seissol_results/` — SeisSol output for comparison

Frontera job script template:

(R-006, R-007 fixes) Follows the established pattern from `jobs/bp5/` — explicit module loads (no conda), `--flag value` CLI matching `seas_bp5_full`:

```
#!/bin/bash
#SBATCH -J tpv102_seas_mfem
#SBATCH -o tpv102_%j.out
#SBATCH -e tpv102_%j.err
#SBATCH -p normal
#SBATCH -N 4
#SBATCH -n 224
#SBATCH -t 02:00:00
#SBATCH -A EAR20006

export LC_ALL=C
export LANG=C
```

```
module load hypre/2.31.0
module load mumps/5.3
module load parmetis
module load fftw3/3.3.8
module list

export LD_LIBRARY_PATH=${TACC_HYPRE_LIB}:${TACC_PARMETIS_LIB}:${TACC_MUMPS_LIB}:${TACC_FFTW3_LIB}:${LD_LIBRARY_PATH

cd /scratch2/10024/zhaochun/seas-project/seas-mfem/miniapps/seas

test -x ./seas_tpv102_driver || { echo "ERROR: driver not built"; exit 1; }

ibrun ./seas_tpv102_driver \
   --mesh tpv102/mesh/tpv102_fine.msh \
   --mesh-scale 1000 \
   --order 2 \
   --bc-mode absorbing \
   --tfinal 12.0 \
   --output-dir tpv102/results/${SLURM_JOB_ID} \
   --output-prefix tpv102
```

Verification metrics (compared against SeisSol + SCEC community):

Metric	Station	Tolerance
Rupture arrival time	(0, -7.5 km)	$<$ 5% of community median
Peak slip rate	(0, -7.5 km)	$<$ 10% of community median
Final slip	(0, -7.5 km)	$<$ 10%
Rupture arrest location	Along-strike	$<$ 1 km of community median
P-wave arrival at surface	(0, 0, 0)	$<$ 0.1 s

3.4.5 Acceptance Criteria (Phase 4b)

- ☐ TPV102 simulation completes 12 s on Frontera without instability or NaN
- ☐ Rupture nucleates at hypocenter within $t < 2$ s
- ☐ Rupture propagates bilaterally and arrests at VW/VS boundary
- ☐ Peak slip rate at station $(0, -7.5$ km) within 10% of SCEC community median
- ☐ Rupture arrival time at station $(0, -7.5$ km) within 5% of community median
- ☐ Free-surface displacement shows correct P-wave and S-wave arrivals
- ☐ `compare_seissol.py` script produces comparison plots
- ☐ All Phase 4a local tests still pass

3.4.6 Dependencies (Phase 4b)

- Depends on: Phase 4a (all local tests passing)
- Required by: Phase 5

3.5 Phase 5: BP5 QD→Dynamic Transfer

3.5.1 Goal

Enable robust bidirectional state transfer between QD (displacement-based) and FD (velocity-stress-based) solvers for hybrid BP5 simulations. The transfer must be fast ($<$ 1% of event duration), introduce no numerical artifacts, and preserve fault state exactly.

3.5.2 5.1 Transfer Speed Analysis

QD→FD transfer involves:

1. **Stress from displacement:** $\sigma = \mathbb{C} : \nabla_s \mathbf{u}$ — element-local, $O(n_e \cdot n_{\text{dof}}^2)$
2. **CG→DG projection:** QD uses CG space (`H1`), FD uses DG space (`L2`). Must project.
3. **Velocity initialization:** Set $\mathbf{v}$ from last QD slip rate.

Cost estimate for BP5 (100k elements, order 2):

Step	Operation	Cost	Time (est.)
Stress computation	Element-local $\nabla \mathbf{u}$	$O(n_e \cdot n_{\text{dof}}^2)$	~0.1 s
CG→DG projection	L^2 projection per element	$O(n_e \cdot n_{\text{dof}}^2)$	~0.1 s
Velocity initialization	Fault DOF loop	$O(n_{\text{fault}})$	~0.001 s
Total QD→FD			**~0.2 s**

A typical dynamic event lasts ~10 s with $\Delta t \sim 10^{-3}$ s = 10,000 time steps. The transfer at 0.2 s is $< 0.002\%$ of event wall time. **Transfer speed is not a concern.**

FD→QD transfer involves:

1. **Accumulated slip:** $\delta_{\text{new}} = \delta_{\text{frozen}} + \int V\, dt$ — tracked during FD phase, $O(n_{\text{fault}})$
2. **QD elasticity solve:** $\mathbf{K}\mathbf{u} = \mathbf{f}(\delta_{\text{new}})$ — one MUMPS solve, $O(n_e^{1.5})$

The MUMPS solve is the most expensive part (~1–5 s for BP5), but it happens once per regime switch. **Not a bottleneck.**

3.5.3 5.2 Numerical Artifact Mitigation

Problem 1: CG→DG stress projection introduces oscillations.

The QD solver uses a continuous Galerkin (CG) space for displacement. The stress $\sigma = \mathbb{C} : \nabla \mathbf{u}$ is one order lower than $\mathbf{u}$ and may have jumps at element boundaries. Projecting to the DG space naively can produce Gibbs-like oscillations.

Solution: Use element-local L^2 projection (no inter-element coupling):

$$\sigma_{\text{DG}}^{(e)} = \mathbf{M}_{\text{DG}}^{(e),-1} \int_{T_e} \Phi_l^{DG}\, \sigma_{\text{CG}}\, dV$$

This is the best L^2 approximation within each element and introduces no new oscillations beyond what σ_{CG} already has.

Problem 2: Velocity initialization mismatch.

In QD, velocity is not an independent variable — it's implicitly zero (quasi-static). Setting $\mathbf{v} = 0$ everywhere in FD creates an initial transient: the fault "sees" zero velocity but nonzero stress, causing a spurious pulse.

Solution: Warm-start velocity from QD slip rate.

At the fault, set $v_{t_1}^+ = +V_{\text{qd},1}/2$ and $v_{t_1}^- = -V_{\text{qd},1}/2$ (and similarly for t_2). In the bulk, set $\mathbf{v} = \mathbf{0}$. This creates a velocity field that is consistent with the current slip rate, minimizing the initial transient.

Additionally, apply a **5-step damped ramp:** during the first 5 FD time steps, linearly ramp the nucleation perturbation from 0 to full strength. This avoids a sharp shock at $t = t_{\text{switch}}$.

Problem 3: Stress equilibrium mismatch.

The QD stress field satisfies the static equilibrium $\nabla \cdot \sigma = \mathbf{0}$. After projecting to DG and adding the velocity field, the FD system may not satisfy dynamic equilibrium $\rho\, \partial \mathbf{v}/\partial t = \nabla \cdot \sigma$. This creates transient waves.

Solution: Equilibrium correction step. After initialization, run 10 FD time steps with artificially high damping ($d = 10\, d_{\text{PML}}$) to absorb transient waves, then switch to normal damping. This "settles" the state.

Problem 4: Fault-tip velocity discontinuity (R-008 fix).

The warm-start sets $v = \pm V_{\text{qd}}/2$ at fault DOFs but $v = 0$ in the bulk. At fault tips (where fault elements meet non-fault elements), this creates a velocity discontinuity that radiates artificial P/S waves. For BP5 with $V_{\text{qd}} \sim 0.1$ m/s at nucleation, this can be significant.

Solution: Gaussian taper near fault tips. Use `FaultGeometry::GetFaultCoords2D()` to identify DOFs near fault tips (within 3 element widths). Apply a smooth taper:

$$v_{\text{init}}(\mathbf{x}) = \frac{V_{\text{qd}}(\mathbf{x})}{2}\, \exp\!\left(-\frac{\text{dist}(\mathbf{x},\, \text{tip})^2}{2\,(3h)^2}\right)$$

where h is the local element size. This smoothly blends fault velocity to zero over ~3 elements, eliminating the discontinuity.

3.5.4 5.3 FD→QD Transfer: Detailed Steps (R-009 Fix)

The FD→QD transfer is not a simple "set slip and solve." The full re-initialization sequence:

1. **Set fault state:** $\delta = \delta_{\text{new}}$, $\psi = \psi_{\text{final,FD}}$ (state variable transfers directly)
2. **Quasi-static elasticity solve:** Call `domain_->Solve(t, slip_bc, displacement)` — full MUMPS solve with the new slip boundary condition
3. **Recompute traction:** Call `domain_->ComputeTraction(displacement, slip, traction)` to get the new QD traction
4. **Verify friction equilibrium:** Check $|\sigma_n\, f(V, \psi) + \eta\, V - \tau| <$ tol at all fault DOFs. If the residual exceeds 10^{-6}, run 1–2 Init-style correction steps (from the 4-phase initialization in CLAUDE.md)
5. **Reset time stepper:** Set QD adaptive dt to a small initial value ($0.01 \cdot L_{\text{nuc}}/V_{\text{nuc}}$) to allow RK45 to find the correct step size

3.5.5 5.4 Unit Tests for Phase 5

#	Test	What	Tolerance
37	`TestQDtoFDStressConsistency`	$\|\sigma_{\text{QD}} - \sigma_{\text{FD}}\|_{L^2} / \|\sigma_{\text{QD}}\|_{L^2}$	$< 10^{-6}$
38	`TestQDtoFDSlipRateContinuity`	$V_{\text{FD}} = V_{\text{QD}}$ at fault	$< 10^{-10}$
39	`TestFDtoQDRoundTrip`	QD→FD→QD (zero evolution) = identity	$< 10^{-12}$
40	`TestFDtoQDSlipAccumulation`	Known $\Delta\delta$ transferred correctly	$< 10^{-12}$
41	`TestTransientSuppression`	After warm-start + damped ramp, transient < 1% of signal (R-008: measured at fault tips, not just center)	$E_{\text{transient}}/E_{\text{signal}} < 0.01$
42	`TestFaultTipTaper`	Velocity taper produces no discontinuity at tips	$\max \|\Delta v\| < 10^{-3}\, V_{\text{qd}}$ at tip elements
43	`TestFDtoQDEquilibrium`	After FD→QD transfer, friction equilibrium residual small (R-009)	$\|\text{residual}\|/\tau < 10^{-6}$

#	Test	What	Tolerance
44	`TestHybridSwitching`	Regime switches at correct V thresholds	Exact transitions

3.5.6 Acceptance Criteria (Phase 5)

- ☐ All 8 regime transfer tests pass (Tests 37–44)
- ☐ QD→FD stress consistency: $\|\sigma_{\text{QD}} - \sigma_{\text{FD}}\|_{L^2} / \|\sigma_{\text{QD}}\|_{L^2} < 10^{-6}$
- ☐ Slip rate continuous across QD→FD transition ($< 10^{-10}$)
- ☐ Round-trip QD→FD→QD (zero evolution) preserves state to $< 10^{-12}$
- ☐ Fault-tip velocity taper eliminates discontinuity ($\max |\Delta v| < 10^{-3}\, V_{\text{qd}}$)
- ☐ After warm-start + damped ramp, transient energy < 1% of signal
- ☐ FD→QD transfer: friction equilibrium residual $< 10^{-6}$ at all fault DOFs
- ☐ `SEASHybridOperator` correctly switches regimes at V_{activate} and $V_{\text{deactivate}}$ thresholds
- ☐ Transfer wall time < 1% of dynamic event duration (estimated ~0.2 s for BP5)
- ☐ Phase 1–4 tests still pass
- ☐ `make test` passes

3.5.7 Dependencies (Phase 5)

- Depends on: Phase 4 (proven FD code), existing QD infrastructure (`SEASQuasiDynamicOperator`)
- Required by: Future hybrid BP5 simulations

3.6 Phase 6: GPU Acceleration

3.6.1 Goal

Accelerate using MFEM GPU infrastructure. Dual friction solver: Brent (CPU), Newton-Raphson (GPU).

3.6.2 Task Breakdown

Task	Description
6.1	CPU profiling baseline (gprof/perf on TPV102 coarse)
6.2	AoS→SoA data layout via `ElementRestriction`
6.3	GPU volume integral kernel (`MFEM_FORALL`)
6.4	GPU face flux kernel
6.5	GPU mass inverse kernel
6.6	**GPU friction kernel: Newton-Raphson with Brent fallback**
6.7	Memory management (`UseDevice`, precompute on device)

3.6.3 Task 6.6: Dual Friction Solver on GPU

```
// GPU kernel: Newton-Raphson with bisection fallback
MFEM_FORALL(i, n_fault_qp, {
   real_t V = data[i].slip_rate;  // warm start from previous step

   // Phase 1: Newton-Raphson (fast, 5 iterations)
   bool converged = false;
   for (int iter = 0; iter < 5; iter++) {
      real_t C = exp(data[i].psi / data[i].a) / (2.0 * V0);
      real_t sinh_arg = V * C;
```

```
        real_t mu_f = data[i].a * asinh(sinh_arg);
        real_t g = sigma_n * mu_f + eta_s * V - Theta;
        if (fabs(g) < 1e-8) { converged = true; break; }
        real_t dmu = data[i].a * C / sqrt(1.0 + sinh_arg * sinh_arg);
        real_t dg = sigma_n * dmu + eta_s;
        V = max(1e-45, V - g / dg);
    }

    // Phase 2: Bisection fallback (guaranteed, 20 iterations)
    if (!converged) {
        real_t V_lo = 0.0, V_hi = Theta / eta_s;
        for (int iter = 0; iter < 20; iter++) {
            real_t V_mid = 0.5 * (V_lo + V_hi);
            real_t g_mid = eval_residual(V_mid, ...);
            if (g_mid > 0.0) V_lo = V_mid; else V_hi = V_mid;
        }
        V = 0.5 * (V_lo + V_hi);
    }

    data[i].slip_rate = V;
});
```

All threads execute the same code path (5 NR iterations, then conditionally 20 bisection iterations). The `if (!converged)` branch is taken by very few threads (< 0.1% for typical parameters), so warp divergence is minimal.

3.6.4 GPU Unit Tests

#	Test	What	Tolerance
43	`TestVolumeIntegralGPUvsCPU`	GPU volume RHS = CPU	$< 10^{-12}$
44	`TestFaceFluxGPUvsCPU`	GPU face flux = CPU	$< 10^{-12}$
45	`TestFullMultGPUvsCPU`	`Mult()` GPU = CPU	$< 10^{-11}$ relative
46	`TestGPUEnergyConservation`	Reflecting box on GPU	$\lvert E_f/E_0 - 1\rvert < 10^{-9}$
47	`TestGPUFrictionNR`	NR on GPU matches Brent on CPU	$< 10^{-8}$ for all test points

3.6.5 Acceptance Criteria (Phase 6)

- ☐ CPU profiling report documents bottleneck breakdown (volume %, face %, fault %, $\mathbf{M}^{-1}$ %, MPI %)
- ☐ All 5 GPU tests pass (Tests 43–47)
- ☐ GPU volume integral matches CPU to $< 10^{-12}$
- ☐ GPU face flux matches CPU to $< 10^{-12}$
- ☐ Full `Mult()` GPU matches CPU to $< 10^{-11}$ relative error
- ☐ GPU energy conservation in reflecting box: $|E_f/E_0 - 1| < 10^{-9}$
- ☐ GPU NR friction solver matches CPU Brent to $< 10^{-8}$
- ☐ $\geq 5\times$ speedup over single-core CPU for TPV102 on NVIDIA A100 or similar
- ☐ Graceful fallback with `Device("cpu")` — all tests pass without GPU
- ☐ Phase 1–5 tests still pass
- ☐ `make test` passes

3.6.6 Dependencies (Phase 6)

- Depends on: Phase 4 (working CPU code to profile and compare against)

- Required by: nothing (optimization phase)

---

4 PART III: CROSS-CUTTING CONCERNS

4.1 Testing Strategy Summary

Phase	Test File	# Tests	Breakdown
1	`test_godunov_flux.cpp`	8	Jacobians, eigenvalues, rotation, flux × 4
1	`test_wave_operator.cpp`	6	Mass inv, P-wave, S-wave, convergence, energy, fault geom
2a	`test_wave_bc.cpp`	7	ABC normal/oblique, free surface × 3, mixed BC, energy decay
2b	`test_pml.cpp`	4	Normal, oblique, energy decay, corner
2c	`test_sbi_boundary.cpp`	(5)	**DEFERRED** — DC mode, single mode, zero reflection, oblique, long-time stability
3	`test_fault_face_flux.cpp`	10	Trial traction × 2, locked/frictionless, continuity, velocity jump, solver reuse, state var, energy, sign
3	`test_friction_solver.cpp`	5	NR convergence, NR derivative, Brent-NR equivalence, extreme psi, hybrid
4a	`test_tpv102_setup.cpp`	5	Boxcar, spatial a, equilibrium, nucleation, stations
4a	`test_tpv102_local.cpp`	6	Nucleation, propagation, free surface, ABC, output, convergence
5	`test_regime_transfer.cpp`	8	Stress consistency, slip rate, round-trip, slip accum, transient, fault-tip taper, FD→QD equilibrium, hybrid switching
6	`test_wave_gpu.cpp`	5	Volume, face, full Mult, energy, GPU friction NR
Active total		**64**	(69 including 5 deferred Phase 2c)

4.2 Risk Assessment

Risk	Likelihood	Impact	Mitigation
PML instability at corners	Medium	High	Test 18b; use split-direction damping; compare with farms_aftershock

Risk	Likelihood	Impact	Mitigation
SBI aliasing from insufficient zero-padding	Medium	Medium	Pad to 2× fault length; Test 17c checks long-time stability
NR fails for extreme ψ/a on GPU	High	Medium	Hybrid NR+bisection fallback (Task 6.6); Test 35 verifies
QD→FD transient contaminates rupture	Medium	High	Warm-start + damped ramp (Section 5.2); Test 41 verifies
Frontera job timeout (TPV102 too slow)	Low	Medium	Profile locally first (Phase 4a); estimate wall time before submitting

4.3 Project Layout After All Phases

```
miniapps/seas/
├── dynamic/
│   ├── wave_state.hpp              Phase 1
│   ├── wave_operator.hpp/.cpp      Phase 1
│   ├── godunov_flux.hpp/.cpp       Phase 1
│   ├── pml_layer.hpp/.cpp          Phase 2b: PML damping
│   ├── sbi_kernel.hpp/.cpp         Phase 2c: FFT traction
│   ├── sbi_boundary.hpp            Phase 2c: SBI BC for WaveOperator
│   ├── fault_face_flux.hpp/.cpp    Phase 3
│   ├── friction_solver.hpp         Phase 3: Brent + NR + Hybrid
│   ├── tpv102_setup.hpp/.cpp       Phase 4
│   ├── regime_transfer.hpp/.cpp    Phase 5
│   ├── seas_dynamic_operator.hpp   Phase 1: coupling operator (R-001)
│   ├── wave_kernels_gpu.hpp/.cpp   Phase 6
│   └── README.md                   Architecture notes
├── config/
│   ├── tpv102_params.hpp           Phase 4
│   └── tpv102.toml                 Phase 4
├── drivers/
│   └── tpv102_driver.cpp           Phase 4
├── solver/
│   ├── seas_hybrid_operator.hpp/.cpp  Phase 5
├── tpv102/
│   ├── mesh/
│   │   ├── tpv102_coarse.geo       Phase 4a (local testing)
│   │   ├── tpv102_medium.geo       Phase 4a
│   │   └── tpv102_fine.geo         Phase 4b (Frontera)
│   ├── scripts/
│   │   ├── frontera_tpv102.sbatch  Phase 4b
│   │   └── compare_seissol.py      Phase 4b
│   └── reference/seissol_results/  Phase 4b
└── tests/
    ├── unit/
    │   ├── test_godunov_flux.cpp       Phase 1: 8 tests
    │   ├── test_wave_operator.cpp      Phase 1: 6 tests
```

```
│   ├── test_wave_bc.cpp           Phase 2a: 7 tests
│   ├── test_pml.cpp               Phase 2b: 4 tests
│   ├── test_sbi_boundary.cpp      Phase 2c: 3 tests
│   ├── test_fault_face_flux.cpp   Phase 3: 10 tests
│   ├── test_friction_solver.cpp   Phase 3: 5 tests
│   ├── test_tpv102_setup.cpp      Phase 4a: 5 tests
│   ├── test_tpv102_local.cpp      Phase 4a: 6 local integration tests
│   ├── test_regime_transfer.cpp   Phase 5: 6 tests
│   └── test_wave_gpu.cpp          Phase 6: 5 tests
└── verification/
    └── tpv102_verification.py     Phase 4b
```

New files: ~30. New LOC estimate: ~5,500. Existing code reused unchanged: ~8,000 LOC. Active unit tests: 64 (plus 5 deferred Phase 2c SBI tests = 69 total when FEM-SBI is implemented).